

■ Design URL Shortener

System Design Interview | Infrastructure / Core Services

algomaster.io | Deep Dive Notes

■ Key Requirements

Functional	Shorten long URL → unique short code; Redirect short URL → original; Custom aliases; Expiry support; Click analytics
Non-Functional	High availability (99.99%); Low latency redirection (<10ms); ~1B URLs, 100:1 read/write ratio
Scale Estimate	1M writes/day → ~12/sec; 100M reads/day → ~1200/sec; Storage: ~500 bytes/URL × 1B = ~500 GB

■ High-Level Architecture

- API Gateway → URL Creation Service (writes) + Redirection Service (reads)
- Load balancer distributes traffic; Redirection service is read-heavy and stateless (easy to scale)
- Primary DB (PostgreSQL/Cassandra) stores URL mappings; Redis cache for hot short codes
- Async worker handles click analytics; CDN can cache redirect responses for popular URLs

■ Short Code Generation Strategies

- **Hash-Based (MD5/SHA256 + Base62):** Hash long URL → take first 7 chars of Base62 encoding; $62^7 \approx 3.5B$ unique URLs; Handle collisions by appending counter
- **Counter-Based:** Global auto-increment counter → Base62 encode; Simple but single point of failure; Use distributed counter shards to scale
- **Pre-generated IDs:** Background job pre-generates codes and stores in 'available' pool; No collision risk at creation time
- **Snowflake IDs:** 64-bit timestamp+machineId+sequence → Base62; Sortable, unique, no coordination needed

■ Deep Dive: Redirection & Caching

- Use HTTP 301 (permanent) for SEO but browser caches it — no analytics; Use 302 (temporary) to always hit server for analytics
- Cache short_code→URL in Redis with TTL matching URL expiry; Cache-aside pattern: check cache first, fallback to DB
- For expired URLs: store expiry_at in DB, TTL in Redis; Background job cleans up expired entries
- Hot URL problem: consistent hashing or local in-memory cache per server to avoid Redis hotspot

■ Database Schema & Indexing

- Table: urls(short_code PK, long_url, user_id, created_at, expires_at, click_count)
- Index on short_code (PRIMARY); Index on user_id for listing user's URLs; Index on expires_at for cleanup jobs

- Use NoSQL (DynamoDB/Cassandra) for pure key-value lookups at massive scale (short_code as partition key)
- Analytics: separate table or stream click events to Kafka → aggregate in data warehouse (Redshift/BigQuery)

■ Trade-offs

SQL (PostgreSQL)	NoSQL (DynamoDB/Cassandra)
✓ Strong consistency	✓ Horizontal scaling native
✓ Easy analytics queries	✓ Sub-millisecond reads
✓ ACID transactions	✓ No joins (denormalized)
✓ Harder to shard at 10B+ scale	✓ Eventual consistency acceptable here

■ Security & Abuse Prevention

- Rate limit URL creation per user/IP (token bucket); Block malicious URLs via safe-browsing API check
- Custom alias validation: block reserved words (/api, /admin), length limits (3-50 chars)
- HTTPS everywhere; Monitor for DDoS patterns; Blocklist known spam domains